1 Preliminary

Condensed notes from references,

- The Implementation of Functional Programming Languages [1]
- Implementing Lazy Functional Languages on Stock Hardware: The Spineless Tagless G-machine [2]

, focusing on abstract machine operational semantics.

Notation:

```
ρ ≡ local env
ρ' = ρ[x ↦ y] ≡ new env with the old env ρ
     plus additional mapping x ↦ y
ρ[x ↦ y] ≡ env ρ with mapping x ↦ y
     present (eg: read the env only)

val ρ σ k = k, k is a literal
val ρ σ k = ρ[k], k is in ρ
val ρ σ k = σ[k], k is in σ
val ρ σ ks = [(val ρ σ k) for k in ks]

ρ a ≡ query env ρ with key a
ρ as ≡ [ρ a for a in as]
h a ≡ query heap h with key(address) a
h as ≡ [h a for a in as]

(vs \n xs ↦ e) ws_f ≡ closure that is non-
     updatable, with parameters xs and free
     variable parameters vs bound with
     values ws_f; note: closure with len(xs)
      > 0 ⟹ it is non-updatable
(vs \u {} ↦ e) ws_f ≡ closure that is
     updatable
```

2 Machine State

a tuple of:

- code
- $as \equiv$ argument stack: for actual arguments of lambdas
- $rs \equiv$ return stack: for continuations
- $us \equiv$ update stack: for updatable closure / thunk
- $h \equiv$ heap: for locals, closures
- $\sigma \equiv$ globals: for top-level variables/binders

initial state:

```
(eval (main {}) {},
 [],
 [],
 [],
 {...; a_i ↦ (vs_i \ π_i xs ↦ e_i) (σ vs_i)},
 {...; main = addr a_main})
```

3 Code Forms

eval e ρ : uses local env. ρ and accesses argument stack

enter a : accesses the argument stack for applying the closure at address a on the heap

ReturnCon c ws : constructor c applied to ws; return it to continuation

ReturnInt k : return primitive type int k to continuation

4 Operational Semantics

state transitions of STG

4.1 eval function application

```
eval (f xs) ρ, as, rs, us, h, σ

where:

  val ρ σ f = addr a

⟹

enter a,
(val ρ σ xs)++as,
rs,
us,
h,
σ

note: ρ is discarded
```

4.2 intermediate state: enter non-updatable closure

```
enter a,
as,
rs,
us,
h[a ↦ (vs \n xs ↦ e) ws_f],
σ

where:
  len(as) ≥ len(xs)

⟹

eval e ρ, as', rs, us, h, σ

(eval body e of closure, in local env ρ)

where:

  ws_a ++ as' = as // partition into 2,
      where ws_a are the actual arguments
      bound to formal parameters xs

  len(xs) = len(ws_a)

  ρ = [vs ↦ ws_f, xs ↦ ws_a]

  (augmented local env., where free
      variables vs are bound to ws_f, xs
      are bound to arguments)

  heap h is unchanged after transition
```

4.3 eval let expression

allocate closures on the heap for let bindings, extend local env with let binding heap addresses, then evaluate the body of the let expression

```
eval (let x1 = vs1 \π1 xs1 ↦ e1; ...; in e)
     ρ,
as,
rs,
us,
h,
σ

⟹

eval e ρ', as, rs, us, h', σ

where:

  ρ' = ρ[x1 ↦ addr a1, ...]

  h' = h[a1 ↦ (vs1 \π1 xs1 ↦ e1)
         (ρrhs vs1);
         ...]

  ρrhs = ρ

note:
  this step converts static variable names
       into runtime values for free variable
        payload via (ρrhs vs1); this is where
        heap allocation occurs

  references to bound variables (to heap
       allocated closures) typically result
       in generation of compiled code to
       access the variables in actual
       implemenation
```

4.4 eval letrec expression

same as eval let expression, except $\rho_{rhs} = \rho$'

4.5 eval case expression

push continuation onto return stack and evaluate scrutinee expression; after evaluation, continue execution at continuation (pick which alternative to execute)

```
eval (case e of alts) ρ, as, rs, us, h, σ

⟹

eval e ρ, as, (alts, ρ):rs, us, h, σ

note: ρ provides context for evaluating
    chosen alternative
```

4.6 eval constructor

eval of an expression that is specifically a constructor

use of an intermediate state: ReturnCon which pops and resumes continuation(s) from return stack

```
eval (c xs) ρ, as, rs, us, h, σ

⟹

ReturnCon c (val ρ σ xs), as, rs, us, h, σ
```

4.7 intermediate state: ReturnCon

choose which alternative to resume execution from the return stack based on matching constructor

```
ReturnCon c ws,
as,
(...; c vs ↦ e; ..., ρ):rs,
us,
h,
σ

⟹

eval e ρ[vs ↦ ws], as, rs, us, h, σ

note:

  args to c are bound to vs in local env

  continuations are pushed onto the return
      stack earlier
```

handling default alternative for ReturnCon:

```
ReturnCon c ws,
as,
(c1 vs1 ↦ e1; ...; default ↦ e_d, ρ):rs,
us,
h,
σ

where:

  c ≠ ci, (∀i)

⟹

eval e_d ρ, as, rs, us, h, σ
```

handling match all variable binder pattern for ReturnCon:

```
ReturnCon c ws,
as,
(c1 vs1 ↦ e1; ...; v ↦ e_d, ρ):rs, us, h, σ

where:

  c ≠ ci, (∀i)

⟹

eval e_d ρ[v ↦ addr a], as, rs, us, h', σ

where:
  // introduce closure for constructor
  h' = h[a ↦ (vs \n {} ↦ c vs) ws]
  vs = newly introduced params/vars
  a = newly introduced address
  len(vs) = len(ws)
```

note: if there is no match to any of the above, then raise error

4.8 eval literal (or any other primitive)

similar logic as constructor; use of intermediate state: ReturnInt / ReturnPrim

```
eval k ρ, as, rs, us, h, σ

⟹

ReturnInt k, as, rs, us, h, σ
```

eval of a variable bound to a primitive value:

```
eval (f {}) ρ[f ↦ Int k], as, rs, us, h, σ

⟹

ReturnInt k, as, rs, us, h, σ
```

4.9 intermediate state: ReturnInt (generalizes to ReturnPrim for any unboxed primitive literal/op)

purpose: search for a continuation on return stack to continue execution

```
ReturnInt k,
as,
(...; k ↦ e; ..., ρ):rs,
us,
h,
σ

⟹

eval e ρ, as, rs, us, h, σ
```

handling alternative with default:

```
ReturnInt k,
as,
(k1 ↦ e1; ...; default ↦ e, ρ):rs,
us,
h,
σ

where:

  k ≠ ki, (∀i)

⟹

eval e ρ, as, rs, us, h, σ
```

handling alternative with match all binder variable:

```
ReturnInt k,
as,
(k1 ↦ e1; ...; x ↦ e, ρ):rs,
us,
h,
σ

where:

  k ≠ ki, (∀i)

⟹

eval e ρ[x ↦ Int k], as, rs, us, h, σ
```

4.10 eval builtin operator

for example, binary operator $\oplus$ with primitive int parameters:

```
eval ⊕{x1, x2}
  ρ[x1 ↦ Int i1, x2 ↦ Int i2],
as,
rs,
us,
h,
σ

⟹

ReturnInt (⊕ i1 i2), as, rs, us, h, σ
```

5 Update Mechanics

this is applicable for updatable closure / thunk

technique: detect that an update needs is required when empty return stack is encountered during constructor return, or when insufficient arguments are on argument stack when entering a function closure

2 stages:

1) updatable closure is entered

```
enter a,
as,
rs,
us,
h[a ↦ (vs \u {} ↦ e) wsf],
σ

note:

  u denotes updatable closure

  only closure with no arguments is ever
      qualified for being updatable

⟹

eval e ρ, [], [], (as,rs,a):us, h, σ

where:

  ρ = [vs ↦ wsf]

  arg. and return stacks are empty

  similar to entering a non-updatable
      closure, except we push items onto
      the update stack
```

2) after eval of closure corresponding to body of original thunk/closure is done

update is triggered via either:

i) value of closure is data constructor (note: closures never evaluate to unboxed/primitive literals, so unboxed values are not stored in closures / thunks, so ReturnInt state is expected to have non-empty return stack and never triggers update):

try pop continuation from return stack, but the stack is empty $\implies$ triggers update

ii) value of closure is a function:

function tries to bind arguments, but the argument stack does not have enough arguments $\implies$ triggers update

5.1 update mechanics: when the updatable closure is a data constructor

evaluation of closure is data constructor: try pop continuation from return stack, but the stack is empty $\implies$ triggering an update

closure referenced by the update frame is updated, where arg. stack and return stack are restored from update frame, and then try again (possibly trigger another update if return stack is still empty... until the stack is non-empty and a continuation is exposed)

```
ReturnCon c ws, [], [], (as_u, rs_u, a_u):
    us, h, σ

note:

  arg. and return stacks are empty

⟹

ReturnCon c ws, as_u, rs_u, us, h_u, σ

(updating closure a_u with a constructor
    closure)

where:

  h_u = h[a_u ↦ (vs \n {} ↦ c vs) ws]

  len(vs) = len(ws)

  vs ≡ newly introduced params/vars

note:

  no closure has a primitive type:
      ReturnInt state won't see any empty
      return stack
```

5.2 update mechanics: when the updatable closure is a partial application

handle the case where there are not enough arguments on the argument stack for the lambda abstraction

```
enter a, as, [], (as_u, rs_u, a_u):us, h, σ

where:

  len(as) < len(xs)

  h a = (vs \n xs ↦ e) ws_f

  len(xs) > 0 ⟹ non-updatable closure

⟹

enter a, as++as_u, rs_u, us, h', σ

(restore arg. and return stacks, update
    closure a_u in the heap, then retry
    entering a)

where:

  h' = h[a_u ↦ (vs++xs1 \n xs2 ↦ e) (ws_f
      ++ as)]

  xs1 ++ xs2 = xs

  len(xs1) = len(as)

  xs1 are transferred to be free variables
      from original formal parameters of
      lambda along with arguments 'as'

  xs2 are remaining formal parameters

  a is referring to the closure that is
      originally produced from updatable
      closure/thunk a_u's body

  h' is updated for future use: what is in
      closure 'a' is inlined into 'a_u'
      plus adjustments for partial
      application

  a_u is the updatable thunk/closure
```

in order to avoid compiling code for the body, e, for every possible partial application

```
(vs++xs1 \n xs2 ↦ e)
```

, an alternative form exists that reuses closure 'a' being entered within the new updated closure:

```
enter a, as, [], (as_u, rs_u, a_u):us, h, σ

where:

  len(as) < len(xs)

  h a = (vs \n xs ↦ e) ws_f

  len(xs) > 0

 ⟹

enter a, as++as_u, rs_u, us, h_u, σ

where:
  h_u = h[a_u ↦ ((f:xs1) \n {} ↦ f xs1) (a
      :as)]

  xs1++xs2 = xs

  len(xs1) = len(as)

  f ≡ newly introduced variable

  ((f:xs1) \n {} ↦ f xs1) ≡ sharable code
      for all partial applications with the
       same number of arguments 'xs1' ⟹
      create a set of such code covering
      different arities that can be used by
       any partial application with
      matching number of arguments
```

References

[1] Simon L. Peyton Jones. *The Implementation of Functional Programming Languages*. Prentice Hall, 1987.

[2] Simon L. Peyton Jones. Implementing lazy functional languages on stock hardware: The Spineless Tagless G-machine. *Journal of Functional Programming*, 2(2):127–202, 1992.